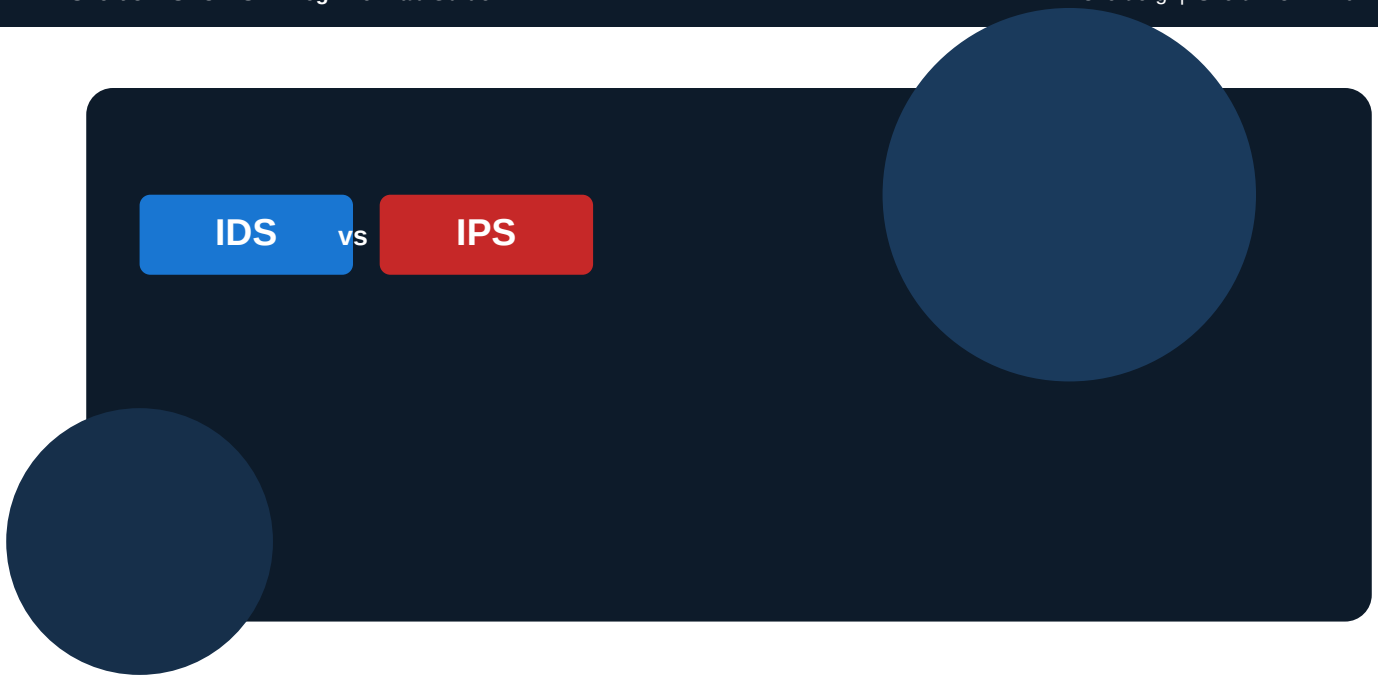


A Beginner's Complete Hands-On Lab Guide

Engine	Scenarios	Level	Date
Snort++ 3.12.2.0	6 Hands-On Labs	Beginner → Intermediate	June 2026

★ Who Is This Guide For?

This guide is written for absolute beginners — no prior Snort or networking experience required.
Every concept is explained from the ground up, with analogies, diagrams, and step-by-step commands.
By the end, you will confidently understand the difference between IDS and IPS and run 6 real labs.
Required: A Linux machine (Ubuntu 20.04+ recommended), Snort 3 installed, and basic terminal use.

Table of Contents

Part 0 — Foundations: What Is Snort 3?

What Is a Network Intrusion System?

Imagine your home has a security guard standing at the front door. That guard watches everyone who enters and exits. Some systems only **watch and report** (like a CCTV camera) — others **actively block** intruders (like a bouncer). Network intrusion systems work exactly the same way for computer network traffic.

What Is Snort 3?

Snort is a free, open-source network security tool created by Cisco. Version 3 (Snort++) is the latest generation, rewritten from scratch for speed and flexibility. It can act as either a detection-only sensor (IDS) or an active traffic blocker (IPS).

Feature	What It Means for You
Multi-threaded Inspection	Snort 3 uses multiple CPU cores simultaneously — faster packet analysis on modern hardware.
Lua Configuration	Instead of old flat config files, Snort 3 uses Lua scripts — more powerful and readable.
DAQ Library (Data Acquisition)	The 'front door' of Snort — it decides HOW packets enter the engine (copy vs. live inline).
AppID / Layer 7 Inspection	Can identify applications by their data patterns, not just port numbers.
Single Shared Rule Table	All threads share one rule set in memory — less RAM, faster lookups.

How Packets Flow Through Snort

Every message sent across a network is broken into small chunks called **packets**. Think of a packet like a single page torn from a letter. Snort reads these packets and compares them against a list of known threat patterns called **rules**.

Packet processing pipeline inside Snort 3

Part 1 — Core Concept: IDS vs IPS Explained Simply

The Analogy You Need to Remember

■ Real-World Analogy

IDS = Security Camera: It watches everything, records it, and alerts the security team — but it NEVER physically stops anyone from walking through.

IPS = Bouncer at the Door: It watches, AND it physically blocks people who match a threat profile. If a packet matches a 'drop' rule, it never reaches the destination.

The Technical Difference: Out-of-Band vs Inline

The single most important difference between IDS and IPS mode in Snort is **WHERE packets physically travel**. This is controlled by the DAQ (Data Acquisition) library — the first component that receives network traffic.

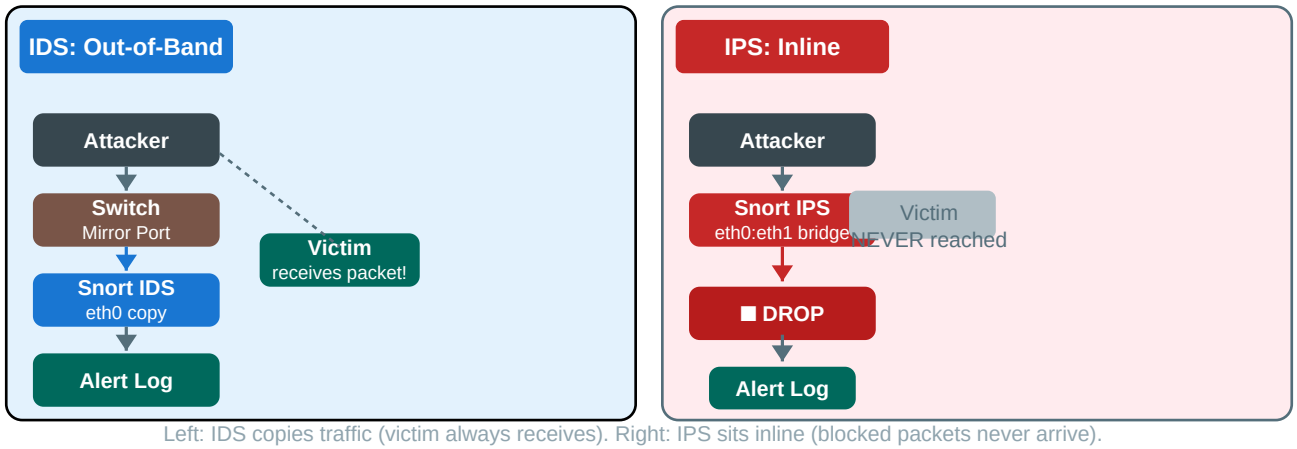
IDS Mode (Out-of-Band)

- Uses a network TAP or switch MIRROR port
- Snort receives a **copy** of each packet
- Real packets flow uninterrupted to destination
- Snort can only **log and alert** — never block
- Zero risk of disrupting legitimate traffic
- Ideal for monitoring without interference

IPS Mode (Inline)

- Snort sits directly **in the network path**
- Every real packet passes through Snort's engine
- Snort can **drop or modify** matching packets
- Blocked packets never reach the destination
- Requires two network interfaces (bridge)
- Ideal for active threat prevention

The Architecture Diagram



Left: IDS copies traffic (victim always receives). Right: IPS sits inline (blocked packets never arrive).

Quick Comparison Table

Property	IDS	IPS
Network position	Out-of-band (copy)	Inline (real path)
Can block packets?	■ No	■ Yes
Rule action keyword	alert	drop
DAQ mode	Default pcap/afpacket passthrough	afpacket with bridge
Risk of disrupting traffic	None	Medium (misconfigured rules)
Best for	Monitoring & forensics	Active prevention
Snort command flag	-i eth0	--daq afpacket -i eth0:eth1
Config setting	(default)	ips = { mode = 'inline' }

Part 2 — Setting Up Your Lab Environment

■ Recommended Lab Setup

Option A (Easiest): Two VirtualBox/VMware VMs on the same Host-Only or Internal network.
- VM1 = Snort sensor (Ubuntu 20.04+, 2 NICs for IPS mode)
- VM2 = Attacker/tester machine (any Linux)

Option B: One physical machine with a second NIC (for IPS inline bridge).
Option C: GNS3 or EVE-NG virtual lab with Snort appliance.

Step-by-Step Installation (Ubuntu 20.04+)

■ Step 1: Update your system packages

Step 1
```
$ sudo apt-get update && sudo apt-get upgrade -y
```
This ensures you have the latest package lists before installing anything.

■ Step 2: Install Snort 3 dependencies

Step 2
```
$ sudo apt-get install -y snort3
```
On Ubuntu 20.04+, Snort 3 is available in the default repositories. For other distros, see: https://snort.org/snort3

■ Step 3: Verify installation

Step 3
```
$ snort --version
```
You should see output like: Snort++ 3.x.x.x If not, check installation logs.

■ Step 4: Locate your config file

Step 4
```
$ ls /etc/snort/snort.lua
```
This Lua file is Snort's main configuration. We'll edit it for IPS mode later.

■ Step 5: Locate rules folder

Step 5
```
$ ls /etc/snort/rules/
$ # You should see: local.rules (our custom rules go here)
```
All 6 lab scenarios add rules to local.rules — never modify built-in rule files.

■ Step 6: Create log directory

Step 6

```
$ sudo mkdir -p /var/log/snort
$ sudo chmod 777 /var/log/snort
```

Snort writes all alerts to this folder. The chmod ensures Snort can write freely.

Understanding Your Network Interfaces

Before running any scenario, identify your network interface names. Modern Linux uses names like **eth0**, **ens33**, or **enp0s3**. Always replace 'eth0' in commands with your actual interface name.

Find your interfaces

```
$ ip link show
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 ...
   # Your interface name is after the number (here: eth0)
$ # For IPS mode you'll need TWO interfaces:
$ # eth0 = incoming side,  eth1 = outgoing side
```

■ Understanding Snort Rule Files

Rules live in /etc/snort/rules/local.rules
Each rule is ONE line in this format:
ACTION PROTOCOL SRC_IP SRC_PORT -> DST_IP DST_PORT (OPTIONS;)
Examples:
alert tcp any any -> any 22 (msg:"SSH detected"; sid:1000001; rev:1;)
drop icmp any any -> any any (msg:"Ping blocked"; sid:1000004; rev:1;)
The SID (Snort ID) must be unique for every rule. Use 1000001, 1000002, etc.

PART I — IDS SCENARIOS (Detection Only)

■ IDS Mode Reminder

In ALL three IDS scenarios below, Snort runs in PASSIVE mode.
Snort reads copies of packets — the real packets ALWAYS reach their destination.
The attacker's ping will succeed. The SSH banner will be shown. The netcat transfer completes.
Snort ONLY writes a log entry. It NEVER blocks anything in IDS mode.
The core launch command for all IDS scenarios:
sudo snort -c /etc/snort/snort.lua -i eth0 -A alert_fast -l /var/log/snort -k none

IDS 1

ICMP Ping Network Scan Detection

Mode: Passive Detection — traffic is NEVER blocked

■ Objective

Passively capture and log ICMP echo request packets (ping), which attackers use to discover live hosts on a network — without disrupting

■ Background: What Is ICMP and Why Do Attackers Ping?

ICMP stands for Internet Control Message Protocol. When you 'ping' an IP address, you send an ICMP Echo Request and wait for an Echo Reply. Attackers use this to quickly find out which IP addresses have live computers behind them — this is called 'network mapping' or 'host discovery'. Detecting it is the first step in identifying a scanning attack.

■ The Snort Rule (Copy This Exactly)

■ Where to add this rule

Open a terminal and run: sudo nano /etc/snort/rules/local.rules
Paste the rule below at the bottom of the file.
Save with Ctrl+O, then Enter. Exit with Ctrl+X.
Each rule must be on a single line — no line breaks inside a rule!

local.rules

```
# Add to /etc/snort/rules/local.rules:
alert icmp any any -> any any (msg:"IDS-1: ICMP Scan Activity Detected"; sid:1000001; rev:1;)
```

■ Rule Anatomy — What Each Part Means:

Part	Value	Meaning
Action	alert	'alert' = log & notify | 'drop' = block & log
Protocol	icmp	Which protocol: tcp, udp, icmp, ip
Source IP	any	'any' = match all source IP addresses
Src Port	any	'any' = match all source ports

Direction	->	'→' = match traffic flowing this direction
Dest IP	any	'any' = match all destination IP addresses
Dest Port	any	Specific port or 'any'

■ Step 1: Start Snort

Start Snort

```
$ sudo snort \
    -c /etc/snort/snort.lua \
    -i eth0 \
    -A alert_fast \
    -l /var/log/snort \
    -k none

# Flags explained:
# -c  = path to config file
# -i  = network interface to listen on (replace eth0 with yours)
# -A alert_fast = write alerts in a compact, fast format
# -l  = directory to write logs
# -k none = skip checksum validation (useful in labs)
```

■ Snort is running — what does that look like?

You will see startup messages, then Snort will show: 'Commencing packet processing'
Leave this terminal window OPEN. Snort runs continuously until you press Ctrl+C.
Open a NEW terminal window to run your test commands below.

■ Step 2: Trigger the Test — ping -c 4 [Target_IP]

Open a second terminal (or use your attacker VM). Run the command below to generate the traffic that Snort will detect:

Attacker: ping -c 4 [Target_IP]

```
$ ping -c 4 192.168.1.100
PING 192.168.1.100: 56 data bytes
64 bytes from 192.168.1.100: icmp_seq=0 ttl=64 time=0.4 ms
64 bytes from 192.168.1.100: icmp_seq=1 ttl=64 time=0.3 ms
64 bytes from 192.168.1.100: icmp_seq=2 ttl=64 time=0.3 ms
64 bytes from 192.168.1.100: icmp_seq=3 ttl=64 time=0.4 ms
--- 192.168.1.100 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss   <-- Success! IDS never blocks
```

■ Step 3: Check the Alert Log

After triggering the traffic, check if Snort logged an alert. Open a third terminal:

Alert Log Output

```
$ cat /var/log/snort/alert_fast.txt

06/06-14:10:02.105 [**] [1:1000001:1] "IDS-1: ICMP Scan Activity Detected" {ICMP} 192.168.1.50 -> 192.1
06/06-14:10:02.205 [**] [1:1000001:1] "IDS-1: ICMP Scan Activity Detected" {ICMP} 192.168.1.50 -> 192.1
06/06-14:10:02.305 [**] [1:1000001:1] "IDS-1: ICMP Scan Activity Detected" {ICMP} 192.168.1.50 -> 192.1
06/06-14:10:02.405 [**] [1:1000001:1] "IDS-1: ICMP Scan Activity Detected" {ICMP} 192.168.1.50 -> 192.1
```

■ What This Output Means:

- [1:1000001:1] = Generator 1, SID 1000001, revision 1 — this identifies which rule fired
- The message in quotes is exactly what you wrote in msg: — this confirms the right rule matched
- {ICMP} = the protocol of the matched packet
- 192.168.1.50 -> 192.168.1.100 = attacker IP to target IP (source to destination)
- You see 4 alert lines — one for each ping packet (we sent -c 4 pings)
- Notice: ping still showed 0% packet loss — IDS mode never blocked anything!

■ What To Look For (Success Checklist)

■ ping command shows 0% packet loss (traffic flows freely)
■ alert_fast.txt contains 4 entries (one per ping)
■ Message matches: 'IDS-1: ICMP Scan Activity Detected'
■ Protocol shows {ICMP}

■ Pro Tip

Try increasing the ping count: ping -c 100 [IP] — you'll see 100 alert entries.
This is exactly how real SOC analysts spot scanning activity in logs.

IDS 2

SSH Brute-Force Probe Detection

Mode: Passive Detection — traffic is NEVER blocked

■ Objective

Detect inbound connection attempts to port 22 (SSH), which indicate an attacker is either scanning for open SSH services or attempting cr

■ Background: What Is SSH and Why Is Port 22 a Target?

SSH (Secure Shell) is the standard way to remotely log into Linux servers using port 22. Attackers scan for open port 22 because if found, they can attempt to guess usernames and passwords (brute-force). Even just CONNECTING to port 22 (before any login attempt) reveals the server's SSH software version in a 'banner' response — this is called 'service fingerprinting'. Detecting these probe connections is critical.

■ The Snort Rule (Copy This Exactly)

■ Where to add this rule

Open a terminal and run: sudo nano /etc/snort/rules/local.rules
Paste the rule below at the bottom of the file.
Save with Ctrl+O, then Enter. Exit with Ctrl+X.
Each rule must be on a single line — no line breaks inside a rule!

local.rules

```
# Add to /etc/snort/rules/local.rules:
alert tcp any any -> any 22 (msg:"IDS-2: Inbound SSH Connection Probed"; sid:1000002; rev:1;)
```

■ Rule Anatomy — What Each Part Means:

Part	Value	Meaning
Action	alert	'alert' = log & notify | 'drop' = block & log
Protocol	tcp	Which protocol: tcp, udp, icmp, ip
Source IP	any	'any' = match all source IP addresses
Src Port	any	'any' = match all source ports
Direction	->	'→' = match traffic flowing this direction
Dest IP	any	'any' = match all destination IP addresses
Dest Port	22	Specific port or 'any'

■ Step 1: Start Snort

Start Snort

```
$ sudo snort \
    -c /etc/snort/snort.lua \
    -i eth0 \
    -A alert_fast \
    -l /var/log/snort \
    -k none

# Flags explained:
# -c  = path to config file
# -i  = network interface to listen on (replace eth0 with yours)
# -A alert_fast = write alerts in a compact, fast format
# -l  = directory to write logs
# -k none = skip checksum validation (useful in labs)
```

■ **Snort is running — what does that look like?**

You will see startup messages, then Snort will show: 'Commencing packet processing'
Leave this terminal window OPEN. Snort runs continuously until you press Ctrl+C.
Open a NEW terminal window to run your test commands below.

■ Step 2: Trigger the Test — curl telnet://[Target_IP]:22

Open a second terminal (or use your attacker VM). Run the command below to generate the traffic that Snort will detect:

Attacker: curl telnet://[Target_IP]:22

```
$ curl telnet://192.168.1.100:22
Trying 192.168.1.100:22...
Connected to 192.168.1.100.
SSH-2.0-OpenSSH_8.4p1 Ubuntu-6ubuntu2    <-- Server banner exposed!
# curl exits after reading the banner. This is exactly what attackers do.
```

■ Step 3: Check the Alert Log

After triggering the traffic, check if Snort logged an alert. Open a third terminal:

Alert Log Output

```
$ cat /var/log/snort/alert_fast.txt

06/06-14:11:15.482 [**] [1:1000002:1] "IDS-2: Inbound SSH Connection Probed" {TCP} 192.168.1.50:54321 -
```

■ What This Output Means:

- The rule action is alert (not drop) — so the connection still succeeds
- {TCP} confirms this is a TCP packet (SSH runs on TCP)
- Source port 54321 is a random high port chosen by the attacker's OS
- Destination port is 22 — matching our rule's -> any 22 target
- The banner 'SSH-2.0-OpenSSH_8.4p1' was visible to the attacker (IDS doesn't block it)

• In a real environment, if you see many of these from one IP, it's likely brute-forcing

■ What To Look For (Success Checklist)

■ curl shows the SSH banner (connection succeeded — IDS mode)
■ alert_fast.txt shows destination port 22
■ Message: 'IDS-2: Inbound SSH Connection Probed'

■ Pro Tip

To simulate brute-forcing, run: for i in {1..20}; do curl telnet://[IP]:22; done
You'll see 20 alert entries — this is the pattern a real IDS would flag for investigation.

IDS 3

Cleartext Sensitive Data Exfiltration Detection

Mode: Passive Detection — traffic is NEVER blocked

■ Objective

Passively inspect TCP payload content to detect sensitive keywords like 'CONFIDENTIAL' crossing the network — simulating data exfiltrat

■ Background: What Is Data Exfiltration and Content Inspection?

Data exfiltration means an attacker (or malicious insider) is secretly sending company data out of the network. When data travels over unencrypted protocols (like plain TCP, HTTP, Telnet), Snort can read the actual content of packets. The 'content' keyword in a Snort rule searches for a specific byte pattern in the packet payload. In a real incident, this catches employees emailing confidential files to personal accounts or attackers using netcat to tunnel data out.

■ The Snort Rule (Copy This Exactly)

■ Where to add this rule

Open a terminal and run: sudo nano /etc/snort/rules/local.rules
Paste the rule below at the bottom of the file.
Save with Ctrl+O, then Enter. Exit with Ctrl+X.
Each rule must be on a single line — no line breaks inside a rule!

local.rules

```
# Add to /etc/snort/rules/local.rules:
alert tcp any any -> any any (msg:"IDS-3: Sensitive Text Pattern Detected"; content:"CONFIDENTIAL"; sid
```

■ Rule Anatomy — What Each Part Means:

Part	Value	Meaning
Action	alert	'alert' = log & notify | 'drop' = block & log
Protocol	tcp	Which protocol: tcp, udp, icmp, ip
Source IP	any	'any' = match all source IP addresses
Src Port	any	'any' = match all source ports
Direction	->	'→' = match traffic flowing this direction
Dest IP	any	'any' = match all destination IP addresses
Dest Port	any	Specific port or 'any'

■ Step 1: Start Snort

Start Snort

```
$ sudo snort \
    -c /etc/snort/snort.lua \
    -i eth0 \
    -A alert_fast \
    -l /var/log/snort \
    -k none

# Flags explained:
# -c  = path to config file
# -i  = network interface to listen on (replace eth0 with yours)
# -A alert_fast = write alerts in a compact, fast format
# -l  = directory to write logs
# -k none = skip checksum validation (useful in labs)
```

■ **Snort is running — what does that look like?**

You will see startup messages, then Snort will show: 'Commencing packet processing'
Leave this terminal window OPEN. Snort runs continuously until you press Ctrl+C.
Open a NEW terminal window to run your test commands below.

■ Step 2: Trigger the Test — echo 'CONFIDENTIAL...' | nc [Target_IP] [Port]

Open a second terminal (or use your attacker VM). Run the command below to generate the traffic that Snort will detect:

Attacker: echo 'CONFIDENTIAL...' | nc [Target_IP] [Port]

```
# On the TARGET machine — listen for incoming data:
$ nc -lvp 4444
Listening on 0.0.0.0 4444

# On the ATTACKER machine — send the data:
$ echo "DATASTREAM: CONFIDENTIAL_COMPANY_RECORDS" | nc -w 2 192.168.1.100 4444

# Target terminal shows the received data:
Connection received from 192.168.1.50
DATASTREAM: CONFIDENTIAL_COMPANY_RECORDS    <-- Data arrived successfully!
```

■ Step 3: Check the Alert Log

After triggering the traffic, check if Snort logged an alert. Open a third terminal:

Alert Log Output

```
$ cat /var/log/snort/alert_fast.txt

06/06-14:12:30.771 [**] [1:1000003:1] "IDS-3: Sensitive Text Pattern Detected" {TCP} 192.168.1.50:55000
```

■ What This Output Means:

- 'content:"CONFIDENTIAL"' searches inside the TCP payload bytes for that exact string
- The match is case-sensitive by default — 'confidential' would NOT trigger this rule

- The data transfer completed successfully — IDS only observed, never blocked
- In a real environment, analysts would investigate: WHO sent it and WHERE it went
- You can extend this rule with: content:"CONFIDENTIAL"; nocase; to catch any capitalization

■ What To Look For (Success Checklist)

■ Netcat transfer completes (data reaches destination)
■ Alert fires with message 'IDS-3: Sensitive Text Pattern Detected'
■ You can verify the port in the log matches where nc was listening

■ Pro Tip

Try changing 'CONFIDENTIAL' to 'PASSWORD' or 'CREDIT_CARD' in the rule to build a DLP sensor.
Add multiple content rules to catch many sensitive patterns simultaneously.

PART II — IPS SCENARIOS (Active Prevention)

■ IPS Mode Reminder — Critical Differences from IDS

In ALL three IPS scenarios, Snort runs INLINE — packets physically pass through it.
You MUST have TWO network interfaces (eth0 and eth1) acting as a bridge.
You MUST add 'ips = { mode = "inline" }' to /etc/snort/snort.lua before starting.
Rules use 'drop' instead of 'alert' — matching packets are DISCARDED immediately.
The attacker's ping will FAIL. SSH probes will TIME OUT. HTTP will be BLOCKED.
The core launch command for all IPS scenarios:
sudo snort -c /etc/snort/snort.lua --daq afpacket -i eth0:eth1 -A alert_fast -l /var/log/snort -k none

IPS 1

ICMP Ping Flood Blocking

Mode: Active Prevention — matching packets are DROPPED

■ Objective

Identify and drop all ICMP echo request packets in real-time, preventing attackers from discovering live hosts via ping scanning or causing

■ Background: Why Block ICMP? The Discovery Prevention Strategy

If an attacker cannot ping your servers, they cannot easily discover which hosts are alive on your network. This is the first line of defense: make assets invisible. In IPS mode, Snort sits between the attacker and the server. When the ping packet arrives on eth0, Snort matches the 'drop' rule and discards the packet — it NEVER exits through eth1 to reach the server. The server has no idea the ping was ever sent.

■ The Snort Rule (Copy This Exactly)

■ Where to add this rule

Open a terminal and run: sudo nano /etc/snort/rules/local.rules
Paste the rule below at the bottom of the file.
Save with Ctrl+O, then Enter. Exit with Ctrl+X.
Each rule must be on a single line — no line breaks inside a rule!

local.rules

```
# Add to /etc/snort/rules/local.rules:
drop icmp any any -> any any (msg:"IPS-1: Ping Dropped Natively"; sid:1000004; rev:1;)
```

■ Rule Anatomy — What Each Part Means:

Part	Value	Meaning
Action	drop	'alert' = log & notify | 'drop' = block & log
Protocol	icmp	Which protocol: tcp, udp, icmp, ip
Source IP	any	'any' = match all source IP addresses
Src Port	any	'any' = match all source ports

Direction	->	'→' = match traffic flowing this direction
Dest IP	any	'any' = match all destination IP addresses
Dest Port	any	Specific port or 'any'

■ Step 1: Start Snort

Start Snort

```
# First, enable inline mode in config file:
$ sudo nano /etc/snort/snort.lua
# Find or add this line inside the file:
#   ips = { mode = 'inline' }
# Save and exit (Ctrl+O, Enter, Ctrl+X)

$ sudo snort \
    -c /etc/snort/snort.lua \
    --daq afpacket \
    -i eth0:eth1 \
    -A alert_fast \
    -l /var/log/snort \
    -k none

# Extra flags for IPS:
# --daq afpacket = use AF_PACKET (Linux fast inline capture)
# -i eth0:eth1  = bridge packets from eth0 to eth1 inline
```

■ Snort is running — what does that look like?

You will see startup messages, then Snort will show: 'Commencing packet processing'
Leave this terminal window OPEN. Snort runs continuously until you press Ctrl+C.
Open a NEW terminal window to run your test commands below.

■ Step 2: Trigger the Test — ping -c 4 [Target_IP]

Open a second terminal (or use your attacker VM). Run the command below to generate the traffic that Snort will detect:

Attacker: ping -c 4 [Target_IP]

```
$ ping -c 4 192.168.1.100
PING 192.168.1.100: 56 data bytes
Request timeout for icmp_seq 0
Request timeout for icmp_seq 1
Request timeout for icmp_seq 2
Request timeout for icmp_seq 3
--- 192.168.1.100 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss   <-- BLOCKED!
```

■ Step 3: Check the Alert Log

After triggering the traffic, check if Snort logged an alert. Open a third terminal:

Alert Log Output

```
$ cat /var/log/snort/alert_fast.txt

06/06-14:15:22.910 [**] [1:1000004:1] "IPS-1: Ping Dropped Natively" {ICMP} 192.168.1.50 -> 192.168.1.1
06/06-14:15:23.910 [**] [1:1000004:1] "IPS-1: Ping Dropped Natively" {ICMP} 192.168.1.50 -> 192.168.1.1
06/06-14:15:24.910 [**] [1:1000004:1] "IPS-1: Ping Dropped Natively" {ICMP} 192.168.1.50 -> 192.168.1.1
06/06-14:15:25.910 [**] [1:1000004:1] "IPS-1: Ping Dropped Natively" {ICMP} 192.168.1.50 -> 192.168.1.1
```

■ What This Output Means:

- 100% packet loss means ZERO ping replies reached the attacker — all dropped
- Snort still logs the dropped packets (drop = block AND log)
- The server never sent a reply because the request never reached it
- This is the key IPS proof: same rule pattern as IDS-1, but 'drop' instead of 'alert'
- Changing one word (alert → drop) completely changes behavior: detect vs. block

■ What To Look For (Success Checklist)

■ Ping shows 100% packet loss (complete blocking confirmed)
■ Alert log still shows entries (drop action logs AND blocks)
■ Server's network interface shows no incoming ICMP traffic (can verify with tcpdump)

■ Pro Tip

To allow your own admin pings but block external ones, change 'any any' to '!192.168.1.0/24 any'.
This is how real firewall rules use Snort — precise source/destination control.

IPS 2

Outbound Malware C2 Communication Blocking

Mode: Active Prevention — matching packets are DROPPED

■ Objective

Prevent internal compromised machines from communicating with external Command & Control (C2) servers by dropping all outbound TC

■ Background: What Is C2 Traffic and Why Block Outbound Telnet?

When malware infects a machine, it often phones home to a Command & Control (C2) server to receive instructions. Port 23 (Telnet) is a clear-text, legacy protocol that modern malware still uses because it's simple. In IPS mode, blocking ALL outbound port 23 connections means even if a machine IS infected, it cannot receive commands from the attacker. This is called 'egress filtering'. Note: the IPS engine sees the outbound SYN packet from the internal machine and drops it before it leaves your network.

■ The Snort Rule (Copy This Exactly)

■ Where to add this rule

Open a terminal and run: sudo nano /etc/snort/rules/local.rules
Paste the rule below at the bottom of the file.
Save with Ctrl+O, then Enter. Exit with Ctrl+X.
Each rule must be on a single line — no line breaks inside a rule!

local.rules

```
# Add to /etc/snort/rules/local.rules:
drop tcp any any -> any 23 (msg:"IPS-2: Outbound Terminal Shell Attempt Blocked"; sid:1000005; rev:1;)
```

■ Rule Anatomy — What Each Part Means:

Part	Value	Meaning
Action	drop	'alert' = log & notify | 'drop' = block & log
Protocol	tcp	Which protocol: tcp, udp, icmp, ip
Source IP	any	'any' = match all source IP addresses
Src Port	any	'any' = match all source ports
Direction	->	'→' = match traffic flowing this direction
Dest IP	any	'any' = match all destination IP addresses
Dest Port	23	Specific port or 'any'

■ Step 1: Start Snort

Start Snort

```
# First, enable inline mode in config file:
$ sudo nano /etc/snort/snort.lua
# Find or add this line inside the file:
#   ips = { mode = 'inline' }
# Save and exit (Ctrl+O, Enter, Ctrl+X)

$ sudo snort \
    -c /etc/snort/snort.lua \
    --daq afpacket \
    -i eth0:eth1 \
    -A alert_fast \
    -l /var/log/snort \
    -k none

# Extra flags for IPS:
# --daq afpacket = use AF_PACKET (Linux fast inline capture)
# -i eth0:eth1  = bridge packets from eth0 to eth1 inline
```

■ Snort is running — what does that look like?

You will see startup messages, then Snort will show: 'Commencing packet processing'
Leave this terminal window OPEN. Snort runs continuously until you press Ctrl+C.
Open a NEW terminal window to run your test commands below.

■ Step 2: Trigger the Test — curl telnet://8.8.8.8:23

Open a second terminal (or use your attacker VM). Run the command below to generate the traffic that Snort will detect:

Attacker: curl telnet://8.8.8.8:23

```
# Try to connect to an external host on port 23:
$ curl telnet://8.8.8.8:23
Trying 8.8.8.8:23...
curl: (28) Failed to connect to 8.8.8.8 port 23: Connection timed out
# The SYN packet was dropped — no connection established
```

■ Step 3: Check the Alert Log

After triggering the traffic, check if Snort logged an alert. Open a third terminal:

Alert Log Output

```
$ cat /var/log/snort/alert_fast.txt

06/06-14:18:05.332 [**] [1:1000005:1] "IPS-2: Outbound Terminal Shell Attempt Blocked" {TCP} 192.168.1.
```

■ What This Output Means:

- Connection timed out — the SYN packet never reached 8.8.8.8, so no SYN-ACK came back
- Snort dropped the very first packet of the TCP handshake (the SYN)

- Without completing the 3-way handshake, no data can ever be exchanged
- The internal machine tried to connect — this attempt itself is logged as evidence
- In an incident response, this log entry tells you: internal IP .100 tried C2 on port 23

■ What To Look For (Success Checklist)

■ curl shows 'Connection timed out' (not 'Connection refused' — it was dropped, not rejected)
■ Alert log shows the outbound SYN packet details
■ No data was transferred to the external server

■ Pro Tip

'Connection timed out' vs 'Connection refused' is important:
TIMEOUT = packet was silently dropped (IPS behavior)
REFUSED = server actively sent back a RST packet (server-side rejection)
IPS dropping gives attackers no feedback — they don't know if the host exists.

IPS 3

Layer 7 HTTP Application-Aware Blocking

Mode: Active Prevention — matching packets are DROPPED

■ Objective

Block HTTP web traffic using Layer 7 Application Identification (AppID), while allowing raw TCP connections on the same port — demonstr

■ Background: What Is Layer 7 / Application-Aware Inspection?

Traditional firewalls block by PORT NUMBER only. But an attacker can run any protocol on any port — they could run HTTP on port 4444 or SSH on port 80 to evade port-based rules. Snort 3's AppID inspects the actual CONTENT and BEHAVIOR of each connection to determine the true application — regardless of port. The 'appids' keyword in a rule tells Snort: 'only match this if the traffic pattern looks like HTTP protocol, no matter what port it's on'. This test proves this by running both real HTTP and raw TCP on port 80.

■ The Snort Rule (Copy This Exactly)

■ Where to add this rule

Open a terminal and run: sudo nano /etc/snort/rules/local.rules
Paste the rule below at the bottom of the file.
Save with Ctrl+O, then Enter. Exit with Ctrl+X.
Each rule must be on a single line — no line breaks inside a rule!

local.rules

```
# Add to /etc/snort/rules/local.rules:
drop tcp any any -> any any (msg:"IPS-3: Layer 7 HTTP Protocol Dropped"; appids:"http"; sid:1000006; re
```

■ Rule Anatomy — What Each Part Means:

Part	Value	Meaning
Action	drop	'alert' = log & notify | 'drop' = block & log
Protocol	tcp	Which protocol: tcp, udp, icmp, ip
Source IP	any	'any' = match all source IP addresses
Src Port	any	'any' = match all source ports
Direction	->	'→' = match traffic flowing this direction
Dest IP	any	'any' = match all destination IP addresses
Dest Port	any	Specific port or 'any'

■ Step 1: Start Snort

Start Snort

```
# First, enable inline mode in config file:
$ sudo nano /etc/snort/snort.lua
# Find or add this line inside the file:
#   ips = { mode = 'inline' }
# Save and exit (Ctrl+O, Enter, Ctrl+X)

$ sudo snort \
    -c /etc/snort/snort.lua \
    --daq afpacket \
    -i eth0:eth1 \
    -A alert_fast \
    -l /var/log/snort \
    -k none

# Extra flags for IPS:
# --daq afpacket = use AF_PACKET (Linux fast inline capture)
# -i eth0:eth1  = bridge packets from eth0 to eth1 inline
```

■ Snort is running — what does that look like?

You will see startup messages, then Snort will show: 'Commencing packet processing'
Leave this terminal window OPEN. Snort runs continuously until you press Ctrl+C.
Open a NEW terminal window to run your test commands below.

■ Step 2: Trigger the Test — Test A: curl -I http://neverssl.com | Test B: nc -v [IP] 80

Open a second terminal (or use your attacker VM). Run the command below to generate the traffic that Snort will detect:

Attacker: Test A: curl -I http://neverssl.com | Test B: nc -v [IP] 80

```
# TEST A — Real HTTP request (WILL be blocked):
$ curl -I http://neverssl.com
curl: (28) Operation timed out after 10000 milliseconds   <-- HTTP BLOCKED ✓

# TEST B — Raw TCP on port 80 (should NOT be blocked):
$ nc -v 192.168.1.100 80
Connection to 192.168.1.100 80 port [tcp/http] succeeded!   <-- RAW TCP ALLOWED ✓
# nc is connected but the server just waits — it's not HTTP traffic
```

■ Step 3: Check the Alert Log

After triggering the traffic, check if Snort logged an alert. Open a third terminal:

Alert Log Output

```
$ cat /var/log/snort/alert_fast.txt

06/06-14:22:10.441 [**] [1:1000006:1] "IPS-3: Layer 7 HTTP Protocol Dropped" {TCP} 192.168.1.50:60200 -
# Notice: Test B (raw nc) produces NO alert entry — Snort identified it as non-HTTP
```

■ What This Output Means:

- Test A (curl) was BLOCKED — curl sends proper HTTP GET/HEAD requests with HTTP headers
- Test B (nc) was ALLOWED — nc just opens a TCP socket, it does NOT send HTTP headers
- Snort's AppID engine analyzed the application behavior, not the port number
- Both used port 80, yet Snort treated them differently — this is Layer 7 intelligence
- This proves Snort 3 is smarter than a basic firewall: it understands WHAT is being sent
- Real use case: block YouTube (HTTP streaming) while allowing your custom app on same port

■ What To Look For (Success Checklist)

■ curl times out / is blocked (HTTP traffic dropped)
■ nc connects successfully (raw TCP is not classified as HTTP — passes through)
■ Only ONE alert entry (for curl) — nc generates no alert
■ This proves application-aware inspection is working

■ Pro Tip

This is the most powerful Snort 3 feature — traditional port-blocking cannot do this.
Real-world use: block Tor browser traffic (appids:"tor"), allow regular HTTPS.

Part 3 — Comparison Summary & Quick Reference

The 6 Scenarios at a Glance

#	Mode	Threat Type	Rule Action	Test Command	Expected Result
IDS-1	IDS	ICMP Ping Scan	alert icmp	ping -c 4 [IP]	Ping succeeds, log written
IDS-2	IDS	SSH Port Probe	alert tcp port 22	curl telnet://[IP]:22	Banner shown, log written
IDS-3	IDS	Data Exfiltration	alert tcp content	nc + echo CONFIDENTIAL	Transfer completes, log written
IPS-1	IPS	ICMP Ping Flood	drop icmp	ping -c 4 [IP]	100% packet loss, log written
IPS-2	IPS	Outbound C2 (port 23)	drop tcp port 23	curl telnet://[IP]:23	Connection timeout, log written
IPS-3	IPS	HTTP Application	drop tcp appids:http	curl http:// vs nc port 80	HTTP blocked; raw TCP allowed

The 3 Most Important Things to Remember

1 1. alert vs drop is the ENTIRE difference

Everything else (the protocol, IPs, ports, options) can be identical. Changing 'alert' to 'drop' transforms an IDS rule into an IPS rule. The pa

2 2. DAQ mode controls in-band vs out-of-band

Without --daq afpacket and -i eth0:eth1, Snort CANNOT drop packets even if you write 'drop' rules. The DAQ library must be in inline mode

3 3. IPS mode requires 'ips = { mode = "inline" }' in snort.lua

This config flag tells Snort's inspection engine to enable inline policy evaluation. Without it, even with the right DAQ, Snort may still operate

Part 4 — Troubleshooting Common Beginner Errors

■ Problem: Snort won't start — 'config file not found'

Check the path: sudo ls /etc/snort/snort.lua
If missing, reinstall: sudo apt-get install --reinstall snort3
Always use the full absolute path in -c /etc/snort/snort.lua

■ Problem: No alerts in alert_fast.txt

Is the log directory writable? Run: sudo chmod 777 /var/log/snort
Is Snort still running? Check with: ps aux | grep snort
Is your interface name correct? Check: ip link show
Is the rule SID unique? Duplicate SIDs cause rules to be silently skipped

■ Problem: IPS mode — packets still not being dropped

Verify snort.lua has: ips = { mode = 'inline' }
Verify you're using: --daq afpacket -i eth0:eth1 (TWO interfaces with colon)
Verify both NICs are UP: ip link show eth0 && ip link show eth1
Are you running as root? Inline mode requires: sudo snort ...

■ Problem: Rule syntax error on startup

Every rule option must end with semicolons: msg:"text"; sid:1000001; rev:1;
No line breaks inside a rule — it must be ONE continuous line
Check quotes: use straight quotes " not curly quotes
Validate syntax only: sudo snort -c /etc/snort/snort.lua --rule-path /etc/snort/rules -T

■ Problem: 'Permission denied' errors

Snort needs root to open raw sockets: always prefix with sudo
Fix log dir: sudo chown -R root:root /var/log/snort && sudo chmod 755 /var/log/snort

■ Problem: AppID (IPS-3) not working — nc also gets blocked

Ensure Snort 3's appid module is loaded in snort.lua: appid = { app_detector_dir = '/usr/local/lib/snort_extra' }
AppID needs a brief 'learning period' — try a few seconds before testing with nc
Verify you're running Snort 3, not Snort 2: snort --version | grep Snort++

Part 5 — Glossary of All Key Terms

Alert	A Snort action that logs a matching packet to the alert file but does NOT block it. Used in IDS mode.
AppID (Application Identification)	Snort 3's ability to identify the true application protocol (HTTP, SSH, etc.) by inspecting packet content patterns, regardless of port number.
C2 (Command and Control)	A server operated by attackers that sends instructions to malware installed on compromised machines.
Content	A Snort rule keyword that searches for a specific byte string in the packet payload: content:"PASSWORD";
DAQ (Data Acquisition Library)	The Snort component that interfaces with the OS to receive packets. Controls whether Snort gets copies (IDS) or live inline packets (IPS).
Drop	A Snort action that DISCARDS the matching packet AND logs it. Only works in IPS inline mode.
Egress Filtering	Blocking outbound traffic from your network (e.g., blocking port 23 outbound to prevent C2 communication).
ICMP	Internet Control Message Protocol. Used by 'ping'. Not a data-carrying protocol but used for network diagnostics and host discovery.
IDS (Intrusion Detection System)	A passive security monitoring system that detects and logs threats but cannot block traffic.
Inline Mode	Snort's IPS configuration where packets physically traverse the Snort engine on their actual path.
IPS (Intrusion Prevention System)	An active security system that can both detect AND block threats in real-time.
Layer 7	The Application Layer of the OSI model — the highest level, where application protocols like HTTP, FTP, DNS operate.
Local.rules	The file (/etc/snort/rules/local.rules) where you add your custom Snort rules.
Mirror Port / SPAN Port	A switch feature that copies all traffic to a dedicated port where Snort (in IDS mode) can observe without being in the data path.
Netcat (nc)	A versatile networking tool for reading/writing raw TCP/UDP data — useful for testing Snort rules.
Out-of-Band	Network architecture where Snort receives traffic copies, not live traffic. Used in IDS mode.
Packet	A small unit of data transmitted over a network. Every file transfer, web page, ping, etc. is broken into packets.
Rev (Revision)	The version number of a Snort rule. Always start at rev:1; and increment when you modify the rule.
Rule	A single line of Snort syntax that defines what traffic to match and what action to take.

SID (Snort ID)	A unique numerical identifier for each Snort rule (e.g., sid:1000001;). Must be unique across all rules.
Snort.lua	Snort 3's main configuration file written in the Lua scripting language. Located at /etc/snort/snort.lua.
TCP (Transmission Control Protocol)	A reliable, connection-oriented protocol used by SSH, HTTP, and most application protocols.
3-Way Handshake	TCP's connection setup: SYN → SYN-ACK → ACK. IPS drops the SYN to prevent this from completing.

End of Snort 3 IDS vs IPS Beginner Lab Guide • Snort++ 3.12.2.0 • June 2026